Instruction Set Architecture

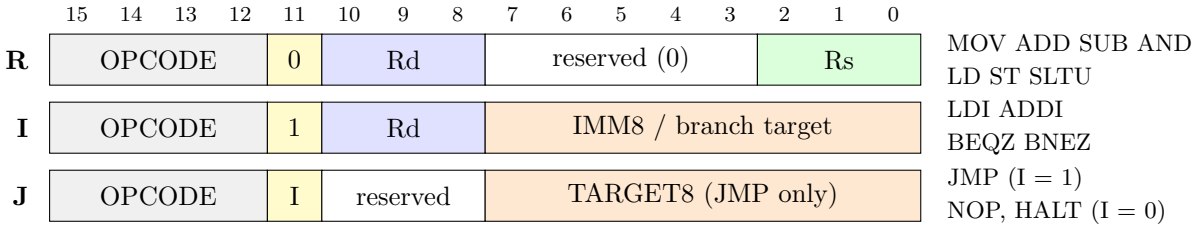
ELL7282 Assignment 2 · Dharun Thota (Entry No. 2026EET2401)

1 Summary

- 16-bit fixed-length instructions, 8-bit data, arithmetic modulo 256.
- Eight registers R0–R7; none is hardwired to zero.
- 8-bit word-addressed PC: starts at 0x00, normally increments by one, wraps from 0xFF to 0x00.
- Three formats (R, I, J); 14 of the 16 primary opcodes are used, 0x1 and 0x2 are reserved.

Field positions never move: the opcode is always [15:12], the destination and first source Rd is always [10:8], the second source Rs is always [2:0], and the 8-bit immediate or branch target is always [7:0]. Bit 11, the I bit, says whether [7:0] is an immediate or [2:0] is a register.

2 Formats



Bit 11 is the I bit: 1 when [7:0] holds an immediate or target, 0 when [2:0] names Rs. The reserved fields are written as 0 and ignored by the hardware (section 4).

- **Rd** [10:8] is register-file read port A (signal `RA`): the destination of every writing instruction, the first source of ADD, SUB, AND, SLTU and ADDI, the store data of ST, and the register tested by BEQZ and BNEZ.
- **Rs** [2:0] is read port B (signal `RB`): the second source, and the address register of LD and ST.
- **IMM8 / TARGET8** [7:0] overlaps Rs. No instruction needs both, so the overlap is safe; the control unit's `USE_A`/`USE_B` flags say which fields are really read (section 6).

3 Opcode map and semantics

Op	Mnemonic	Fmt	Semantics	Reads	Writes
0x0	NOP	J	no state change except PC + 1	–	–
0x1, 0x2	reserved	–	behave exactly as NOP	–	–
0x3	ST Rd, [Rs]	R	DATA[Rs] ← Rd	Rd, Rs	memory, I/O
0x4	BEQZ Rd, target	I	if Rd = 0 then PC ← target	Rd	PC
0x5	BNEZ Rd, target	I	if Rd ≠ 0 then PC ← target	Rd	PC
0x6	JMP target	J	PC ← target	–	PC
0x7	HALT	J	stop fetching; kill younger instructions	–	halt state
0x8	ADDI Rd, imm8	I	Rd ← (Rd + imm8) mod 256	Rd	Rd
0x9	LDI Rd, imm8	I	Rd ← imm8 (bit pattern, no extension)	–	Rd
0xA	MOV Rd, Rs	R	Rd ← Rs	Rs	Rd
0xB	LD Rd, [Rs]	R	Rd ← DATA[Rs]	Rs	Rd
0xC	ADD Rd, Rs	R	Rd ← (Rd + Rs) mod 256	Rd, Rs	Rd
0xD	SUB Rd, Rs	R	Rd ← (Rd − Rs) mod 256	Rd, Rs	Rd
0xE	AND Rd, Rs	R	Rd ← Rd AND Rs	Rd, Rs	Rd
0xF	SLTU Rd, Rs	R	Rd ← 1 if Rd < Rs (unsigned), else 0	Rd, Rs	Rd

Branch and jump targets are absolute 8-bit instruction-word addresses; there are no delay slots. ADDI with 0xFF adds −1 in two's complement. The assembler writes a branch target as a label and emits the absolute address. In BEQZ and BNEZ the register the statement calls Rs is encoded in [10:8].

4 Reserved and unused bits

- **R format, [7:3]:** reserved. The assembler writes 0; the hardware ignores them (they reach nothing except as part of IMM, which R-format instructions do not use).
- **J format:** [10:8] reserved and ignored. NOP and HALT ignore all of [11:0]; JMP uses [7:0] only.
- **The I bit** only chooses the ALU's second operand: B = IMM8 when I = 1, B = R[Rs] when I = 0. A word whose I differs from the assembler's choice still has one defined meaning: ADD, SUB, AND, SLTU with I = 1 use IMM8; MOV with I = 1 is LDI; LDI with I = 0 is MOV; ADDI with I = 0 is ADD; LD and ST with I = 1 use IMM8 as the address. NOP, HALT, BEQZ, BNEZ, JMP and the reserved opcodes ignore I.
- **Reserved opcodes 0x1 and 0x2:** every enable (register write, load, store, branch, HALT) is decoded from specific opcode bits and is 0 for these two, so they execute as NOP with no register, memory or I/O write. The all-zero word is NOP, so unprogrammed ROM runs as NOPs.

5 Machine-code examples

Format	Instruction	Word	Binary: OP I Rd [7:3] [2:0]
R	`ADD R2, R3`	0xC203	1100 0 010 00000 011
R	`LD R1, [R2]`	0xB102	1011 0 001 00000 010
R	`ST R1, [R4]`	0x3104	0011 0 001 00000 100
R	`SLTU R1, R4`	0xF104	1111 0 001 00000 100
R	`MOV R5, R6`	0xA506	1010 0 101 00000 110
I	`LDI R1, 0x64`	0x9964	1001 1 001 01100 100
I	`ADDI R1, 0xFF`	0x89FF	1000 1 001 11111 111
I	`BEQZ R1, 0x20`	0x4920	0100 1 001 00100 000
J	`JMP 0x20`	0x6820	0110 1 000 00100 000
J	`HALT`	0x7000	0111 0 000 00000 000
J	`NOP`	0x0000	0000 0 000 00000 000

A word with junk in the reserved bits behaves like its clean twin: ADD R1, R2 is 0xC102, and with [7:3] all set it is 0xC1FA, which executes identically.

6 Why this encoding

The encoding was chosen so that decoding needs almost no logic.

- **Fixed fields.** Every field always sits in the same bits, so DECODE is pure wiring. Registers are read from [10:8] and [2:0] while the opcode is still being decoded, and the hazard unit always compares the same bits.
- **Two-operand form.** Rd ← Rd op Rs needs only two 3-bit register fields. That leaves [7:0] free for a full 8-bit immediate or target, and the write-back address is always [10:8].
- **Opcodes grouped by high bits.**
  - 0x8–0xF are exactly the register writers, so `isRW` $= o_3$ with no gate.
  - 0x4–0x7 are control flow ($\overline{o_3} o_2$); $o_1 o_0$ picks BEQZ, BNEZ, JMP or HALT.
  - LD (0xB) and ST (0x3) differ only in o_3, so one term $\overline{o_2} o_1 o_0$ finds a memory access.
  - In 0xC–0xF, $o_1 o_0$ selects the ALU operation. ADDI (0x8) shares 00 with ADD, so one `isAdd` term serves both.
- **I bit.** It drives the ALU's B-operand mux directly as `isImm`.
- **Safe defaults.** NOP is 0x0000, so an empty ROM word or a pipeline bubble is harmless. The reserved opcodes 0x1 and 0x2 match no enable term, so they act as NOP with no extra logic.

7 Decoder and control word

DECODE only slices the word: `OPCODE` = [15:12], `I` = [11], `RA` = [10:8], `IMM` = [7:0], `RB` = [2:0]. CTRL derives every control signal from OPCODE ($o_3o_2o_1o_0$) and I with gates (JMP sets both branch flags, so it is always taken):

$$\text{is_ctrl} = \overline{o_3}\,o_2, \qquad \text{mem} = \overline{o_2}\,o_1o_0$$

$$\texttt{isRW} = o_3, \quad \texttt{isImm} = I, \quad \texttt{isLD} = \text{mem}\,o_3, \quad \texttt{isST} = \text{mem}\,\overline{o_3}$$

$$\texttt{isBEQZ} = \text{is_ctrl}\,\overline{o_0}, \quad \texttt{isBNEZ} = \text{is_ctrl}\,(o_1 \oplus o_0), \quad \texttt{isHALT} = \text{is_ctrl}\,o_1o_0$$

$$\texttt{isPass} = \overline{o_2}(o_1 + o_0), \quad \texttt{isAdd} = \overline{o_1}\,\overline{o_0}, \quad \texttt{isSub} = o_2\overline{o_1}o_0$$

$$\texttt{isAnd} = o_2o_1\overline{o_0}, \quad \texttt{isSLTU} = o_2o_1o_0$$

$$\texttt{USE_A} = o_2(o_3 + \overline{o_1}) + o_3\overline{o_1}\,\overline{o_0} + \overline{o_3}\,\overline{o_2}\,o_1o_0$$

$$\texttt{USE_B} = \overline{I}\,(o_3 + \text{mem})$$

For NOP, the reserved opcodes, the branches, JMP and HALT the ALU still produces some result (marked * in the table), but it is never used: `isRW`, `isLD` and `isST` are all 0. In EX, the branch decision is `isBranchTaken` = (`isBEQZ` AND zero) OR (`isBNEZ` AND NOT zero), where zero tests the forwarded Rd value.

Op	Bits	Mnemonic	I	USE_A	USE_B	isRW	isImm	ALU op	isLD	isST	isBEQZ	isBNEZ	isHALT
0x0	0000	NOP	0	0	0	0	0	(add)*	0	0	0	0	0
0x1	0001	(reserved)	0	0	0	0	0	(pass)*	0	0	0	0	0
0x2	0010	(reserved)	0	0	0	0	0	(pass)*	0	0	0	0	0
0x3	0011	ST	0	1	1	0	0	pass	0	1	0	0	0
0x4	0100	BEQZ	1	1	0	0	1	(add)*	0	0	1	0	0
0x5	0101	BNEZ	1	1	0	0	1	(sub)*	0	0	0	1	0
0x6	0110	JMP	1	0	0	0	1	(and)*	0	0	1	1	0
0x7	0111	HALT	0	0	0	0	0	(sltu)*	0	0	0	0	1
0x8	1000	ADDI	1	1	0	1	1	add	0	0	0	0	0
0x9	1001	LDI	1	0	0	1	1	pass	0	0	0	0	0
0xA	1010	MOV	0	0	1	1	0	pass	0	0	0	0	0
0xB	1011	LD	0	0	1	1	0	pass	1	0	0	0	0
0xC	1100	ADD	0	1	1	1	0	add	0	0	0	0	0
0xD	1101	SUB	0	1	1	1	0	sub	0	0	0	0	0
0xE	1110	AND	0	1	1	1	0	and	0	0	0	0	0
0xF	1111	SLTU	0	1	1	1	0	sltu	0	0	0	0	0

I is shown as the assembler writes it. `USE_A` and `USE_B` mark the register fields an instruction really reads; the hazard and forwarding logic use them so that an immediate, a jump target or a destination never causes a false stall.

8 Custom instruction: SLTU (set if less than, unsigned)

Encoding. R format, opcode 0xF: `1111 0 ddd 00000 sss`. Example: `SLTU R1, R4` = 0xF104.

Semantics. Rd ← 0x01 if Rd < Rs as unsigned 8-bit numbers, else 0x00. Rs is unchanged. With I = 1 it compares against IMM8.

Why it helps the control program. The program must detect WATER_LEVEL $\geq$ target and WATER_LEVEL $\leq$ threshold even when readings skip past the threshold, and the HEAVY target (180) is above 127. Equality tests fail on a skip, and SUB alone cannot tell $200 - 180$ from $20 - 180$ modulo 256. SLTU makes each test one instruction:

- Fill, loop while level < target:
  `fill_loop:  LD R1, [R6]; SLTU R1, R4; BNEZ R1, fill_loop`
- Drain, loop while threshold < level:
  `drain_loop:  LD R1, [R6]; MOV R2, R4; SLTU R2, R1; BNEZ R2, drain_loop`

A software-only unsigned compare would need a test of the most significant bits and a subtraction, about 8–10 instructions per check.

Datapath change. None outside the ALU. The ALU already computes X − Y as X + NOT Y + 1 for SUB. For SLTU it runs the same sum; the adder's carry-out is 1 exactly when X $\geq$ Y, so the result is bit 0 = NOT COUT, bits 7:1 = 0. Cost: one inverter, one extra input on the ALU's result mux, and one control signal (`isSLTU`, bit 4 of `aluSignals`).

Pipeline timing and hazards. Like every ALU instruction: operands needed in EX, result at the end of EX, written in WB. Both operands are forwarded from EX/MEM and MEM/WB; a load feeding it needs the usual single bubble (`USE_A` = `USE_B` = 1). There are no condition flags, so it creates no flag hazard, and the branch that follows it is ordinary BNEZ forwarding.